

1.4. Podział prac

Projekt MoodFlow został zrealizowany przez dwuosobowy zespół autorski w ramach wieloautorskiej pracy inżynierskiej. Z uwagi na rozmiar aplikacji obejmującej warstwę serwerową, dwie odrębne aplikacje frontendowe (panel pracownika oraz panel administracyjny dla działu HR i właściciela platformy), bazę danych, infrastrukturę konteneryzacji oraz dokumentację techniczną, podział prac był koniecznością wynikającą z zakresu projektu. Jednocześnie obaj autorzy traktowali projekt jako sposób na doskonalenie swoich umiejętności w obszarze inżynierii oprogramowania oraz zdobycie doświadczenia w wytwarzaniu nowoczesnych aplikacji internetowych, w związku z czym wiele zadań realizowano wspólnie, w trybie programowania w parze (*pair programming*) lub wzajemnego przeglądu kodu (*code review*).

Podział prac został zaprojektowany w sposób zapewniający równomierne obciążenie oraz pokrycie wszystkich obszarów projektu, jednocześnie pozwalający każdemu z autorów na zdobycie głębszej wiedzy w wybranej specjalizacji. Pierwszy autor pełnił rolę liderką w obszarze warstwy serwerowej, infrastruktury oraz bezpieczeństwa aplikacji, drugi autor — w obszarze warstwy prezentacji obu aplikacji frontendowych oraz testowania manualnego. Oba obszary były jednak współrealizowane — przy implementacji nowych funkcjonalności obejmujących pełen przepływ danych (np. wypełnienie testu, generowanie raportu) zadania były dzielone tak, aby każdy z autorów uczestniczył w pracach nad backendem i frontendem. Oba obszary dokumentacji oraz redakcji pracy inżynierskiej były realizowane wspólnie.

Procentowy udział poszczególnych etapów pracy projektowej oraz wkład każdego z autorów przedstawia Tabela 1.1. Wartości zostały oszacowane na podstawie liczby tygodni poświęconych na poszczególne obszary, liczby commitów w systemie kontroli wersji oraz złożoności realizowanych zadań. Łączny udział każdego z autorów w projekcie wyniósł 50%.

Tabela 1.1 Podział zadań w projekcie z procentowym udziałem każdego z autorów (źródło: opracowanie własne)

Etap pracy projektowej	[Autor 1]	[Autor 2]	Razem
Analiza dziedziny, badanie rynku, analiza SWOT, sporządzenie wymagań funkcjonalnych i niefunkcjonalnych metodą MoSCoW	4%	4%	8%
Projektowanie architektury systemu, modelu danych (ERD), diagramów UML i BPMN, dobór stosu technologicznego	5%	2%	7%
Implementacja warstwy serwerowej (NestJS, TypeORM, JWT, autoryzacja, multi-tenant, scoring testów psychologicznych, audit log)	18%	7%	25%
Implementacja aplikacji pracownika (React, Vite, Redux Toolkit, Tailwind CSS, PWA, tryb jasny i ciemny)	6%	14%	20%
Implementacja panelu administracyjnego dla HR, administratora firmy i właściciela platformy (Next.js, Tailwind CSS, Recharts)	5%	12%	17%

Etap pracy projektowej	[Autor 1]	[Autor 2]	Razem
Konfiguracja bazy danych PostgreSQL, modelowanie schematu, optymalizacja zapytań, anonimizacja agregatów	3%	1%	4%
Procedury testowania manualnego (klikalne i nieklikalne), weryfikacja poprawności scoringu, anonimizacji oraz multi-tenant izolacji	2%	3%	5%
Konteneryzacja (Docker, Docker Compose), konfiguracja środowiska produkcyjnego, wdrożenie na platformie chmurowej Railway, certyfikaty TLS	5%	1%	6%
Sporządzenie dokumentacji technicznej oraz redakcja pracy inżynierskiej	2%	6%	8%
Razem	50%	50%	100%

Do organizacji pracy oraz zarządzania kodem źródłowym wykorzystywano system kontroli wersji **Git** [1] z hostingiem repozytorium na platformie **GitHub** [2]. Każda zmiana była dokumentowana czytelnym komunikatem *commit*, a kluczowe decyzje techniczne wpisywane do dziennika decyzji architektonicznych w katalogu docs/ repozytorium. Praca nad kodem odbywała się równolegle na osobnych gałęziach (*feature branches*), które po zakończeniu zadania były scalane do gałęzi głównej po wzajemnym przeglądzie kodu. Przyjęty model współpracy umożliwił zachowanie przejrzystości historii zmian oraz wzajemną weryfikację jakości kodu.

Środowiskiem rozwojowym był edytor **Visual Studio Code** [3] z dodatkami ESLint, Prettier oraz Tailwind CSS IntelliSense. W pracy nad interfejsem aplikacji autorzy wspomagali się programem **Pencil** [4] dla głównych ekranów oraz narzędziem **Figma** [5] dla wybranych elementów wymagających szczegółowego prototypowania. Diagramy UML oraz BPMN zostały przygotowane w języku **Mermaid** [6], co pozwoliło na wersjonowanie ich razem z kodem aplikacji. Komunikacja w zespole oraz konsultacje z promotorem realizowane były z wykorzystaniem komunikatora **Discord** [7] oraz platformy **Google Meet** [8], a redakcja pracy inżynierskiej została przeprowadzona w środowisku **Microsoft Word** [9].